



Palina Dubatouka

Reinforcement Learning for Depth-Based Obstacle Avoidance

A Gazebo Simulation of the Pio Mobile Robot

BACHELOR THESIS

submitted in fulfillment of the requirements for the degree of

BACHELOR OF SCIENCE (BSC)

Programme: Bachelor's programme Robotics and Artificial Intelligence
Branch of study: Autonomous Systems and Robotics

UNIVERSITÄT KLAGENFURT

Evaluator
Prof. Jan STEINBRENER
Institute of Smart Systems Technologies
Control of Networked Systems

Klagenfurt am Wörthersee, September 18, 2026

Declaration of Authorship

I hereby declare in lieu of an oath that

- the submitted academic paper is entirely my own work and that no auxiliary materials have been used other than those indicated,
- I have fully disclosed all assistance received from third parties during the process of writing the thesis, including any significant advice from supervisors,
- any contents taken from the works of third parties or my own works that have been included either literally or in spirit have been appropriately marked and the respective source of the information has been clearly identified with precise bibliographical references (e.g. in footnotes),
- to date, I have not submitted this paper to an examining authority either in Austria or abroad and that
- when passing on copies of the academic thesis (e.g. in bound, printed or digital form), I will ensure that each copy is fully consistent with the submitted digital version.
- I have fully and truthfully declared the use of generative models (Artificial Intelligence, e.g. ChatGPT, Claude, NotebookLMs) including the product version.

I understand that the digital version of the academic thesis submitted will be used for the purpose of conducting a plagiarism assessment.

I am aware that a declaration contrary to the facts will have legal consequences.

Palina Dubatouka m.p. Klagenfurt am Wörthersee, September 18, 2026

Abstract

Training a model for obstacle avoidance in a closed space requires addressing multiple challenges in robot modelling, simulation setup, and policy learning. In this thesis, the Pio robot's simulation model was corrected and set up in Gazebo. Its obstacle-avoidance behaviour was then learned with tabular Q-learning, using a state derived from a discretised depth-camera image and a three-action control space (forward, turn left, turn right). The resulting policy was trained in five room configurations, with the robot assigned a random starting pose in each episode. Further experiments tested whether the policy generalised to previously unseen conditions. Testing the robot in ten unseen rooms with three trials per room showed that it completed 24 of the 30 trials without a collision, although its performance varied with the starting pose and room layout. Analysis of the learned policy showed that, although the reward function treats left and right turns symmetrically, the policy preferred right turns while remaining capable of turning in both directions. These results show that tabular Q-learning with a discretised depth representation can provide effective local obstacle avoidance. However, the remaining limitations were most apparent in narrow passages and situations requiring an early, direction-specific response.

Contents

Affidavit iii

Abstract v

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Aim of the Thesis . . . 1
- 1.3 Thesis Structure . . . 2

2 Background and Theory 3
- 2.1 Robot Description and Modelling . . . 3
  - 2.1.1 URDF . . . 3
  - 2.1.2 SDF . . . 4
  - 2.1.3 XML Macros (Xacro) . . . 4
- 2.2 ROS and Gazebo . . . 4
  - 2.2.1 Robot Operating System . . . 4
  - 2.2.2 Gazebo . . . 5
  - 2.2.3 The ROS-Gazebo Communication Loop . . . 5
- 2.3 Sensor and State Information . . . 6
- 2.4 Navigation and Obstacle Avoidance . . . 7
- 2.5 Reinforcement Learning . . . 7

3 Related Work 9

4 System Setup and Implementation 13
- 4.1 Software and Hardware Overview . . . 13
- 4.2 Robot Model Preparation . . . 14
- 4.3 Rear Caster Wheel Implementation . . . 15
- 4.4 Drive and Motion Control . . . 16
- 4.5 Odometry Setup . . . 17
- 4.6 Camera Integration . . . 17
- 4.7 Simulation Environment . . . 18

5 Reinforcement Learning Approach 21
- 5.1 Implementation Details . . . 21
  - 5.1.1 Depth-Image Discretization . . . 21
  - 5.1.2 Action Space . . . 22
  - 5.1.3 Reward Function . . . 23
  - 5.1.4 Learning Algorithm . . . 23
- 5.2 Training Procedure . . . 24
  - 5.2.1 Checkpoint Selection . . . 24
  - 5.2.2 Learned Policy . . . 25

6 Experimental Evaluation and Discussion **27**
6.1 Experimental Setup 27
6.2 Evaluation Metrics 27
6.3 Qualitative Results 27
6.3.1 Successful Trials 28
6.3.2 Occasional Collisions 28
6.3.3 Consistent Failure in Room 4 28
6.4 Discussion 30

7 Conclusion **31**
7.1 Summary 31
7.2 Limitations 31
7.3 Future Work 31

Bibliography **33**

List of Figures

2.1 Link-Joint Coordinate Frames [8]. 4

4.1 Front and side views of the Pio robot model and its main components 13
4.2 `pio_description` ROS package. 14
4.3 Comparison between the initial URDF model and the complete Fusion assembly used to identify the missing caster wheel 16
4.4 RGB and depth images 18
4.5 Obstacles present in the environment 19
4.6 Custom rooms used for training 19
4.7 Custom rooms used for testing and evaluating 20

5.1 Collision-free training episodes per 100-episode block 25

6.1 Successful trajectories from nine test rooms 29
6.2 Failed trajectories in Rooms 2 and 5 29
6.3 Failed trajectories in Room 4 29

List of Tables

5.1 Discrete action space used for training and evaluation. 23
5.2 Q-learning hyperparameters. 24
5.3 Learned Q-values for all represented states 25

6.1 Evaluation results for the ten test rooms. 28
6.2 Overall evaluation results and action distribution 28

Chapter 1

Introduction

1.1 Motivation

Nowadays, mobile robots are used in a wide range of applications, including logistics, healthcare, inspection, assistance, and autonomous transportation. Depending on their intended purpose, these robots may differ in their mechanical configuration, sensing capabilities, control methods, and level of autonomy. Developing and testing autonomous robotic systems directly on physical hardware can be expensive, time-consuming, and potentially unsafe, particularly during the early stages of development, when navigation and obstacle-avoidance algorithms may still produce unexpected behaviour.

Computer simulation provides a safe, repeatable, and cost-effective environment in which robot models, sensors, controllers, and navigation algorithms can be developed and evaluated before being deployed on a physical platform [1]. It also enables experiments to be repeated under controlled conditions and allows different configurations and scenarios to be tested without risking damage to the robot or its surroundings [2]. When the simulation model and software architecture closely represent the physical system, parts of the developed software can often be transferred to the real robot with only minor modifications [2], [3].

This thesis focuses on the mobile robot Pio. The motivation of the project is to create a stable and sufficiently realistic simulation of the physical robot using Gazebo integrated with ROS. The simulation includes the robot's mechanical structure, motion behaviour, sensors, and interaction with an indoor environment. It provides a platform for developing and evaluating autonomous navigation, perception, and obstacle-avoidance methods before their deployment on the physical robot.

By first validating these components in simulation, the development process becomes safer, more systematic, and easier to reproduce. The resulting simulation environment also provides a foundation for training and evaluating reinforcement-learning methods, including Q-learning-based navigation, under different obstacle configurations and operating conditions [3], [4], [5].

1.2 Aim of the Thesis

Although an initial URDF model of the Pio mobile robot was available, it was not directly suitable for dynamic simulation in Gazebo. Several physical, kinematic, visual, and sensor-related properties were either missing or incorrectly configured, which led to unstable robot behaviour, limited controllability, and improper sensor coordinate frames.

Therefore, the primary aim of this thesis is to establish a stable 3D simulation of a mobile robot within a virtual environment, and to utilize this platform to deploy and evaluate a Reinforcement Learning agent for autonomous indoor navigation.

To achieve this, the project is structured around two main high-level objectives:

1. Reconstruct and configure the robot's Unified Robot Description Format (URDF) model so that its simulated visuals, coordinate frames, and physical behaviors accurately correspond to those of the real-world hardware [6]. This involves correcting structural instabilities and kinematic omissions in the preliminary robot description to establish a reliable dynamic foundation within the simulator [1].

2. Implement and successfully apply a model-free Reinforcement Learning algorithm, specifically Q-learning, to enable the robot to perform local obstacle avoidance in a simulated indoor environment [3], [4]. The performance of the trained navigation model was evaluated through 30 300-second (five-minute) trials in ten unseen virtual room configurations, with the aim of navigating each trial without collision. The evaluation also included the analysis of relevant performance metrics, such as the number of collisions per trial, the collision-free rate, distance travelled, and cumulative reward.

1.3 Thesis Structure

The remainder of this thesis is structured as follows.

Chapter 1 introduces the motivation, aim, and structure of this thesis.

Chapter 2 introduces the theoretical background required to understand mobile robot simulation, ROS, Gazebo, URDF models, odometry, and reinforcement learning.

Chapter 3 reviews related work on reinforcement learning in robotics, comparing tabular Q-learning and deep Q-network approaches across mobile-robot obstacle avoidance, manipulation, motor-skill learning, and legged locomotion.

Chapter 4 presents the system setup and implementation of the Pio robot simulation, including the robot model, sensors, odometry, and environment configuration.

Chapter 5 describes the reinforcement learning approach and the design choices for autonomous obstacle avoidance.

Chapter 6 presents the experimental setup and the tests performed to validate the simulated robot, together with the results, evaluation, and discussion of the findings.

Finally, Chapter 7 concludes the thesis, names the limitations, and outlines possible directions for future work.

Chapter 2

Background and Theory

2.1 Robot Description and Modelling

Before a mobile robot can be deployed or evaluated in a virtual workspace, its physical and behavioural characteristics must be systematically described in a language that the simulation software can parse. This process is defined as robot modelling, and it entails representing the physical structure, kinematics, and rigid-body dynamics of the robot. Several simulators, including Gazebo, Webots, and Unity, can be used for this purpose. These simulators use structured robot description formats to represent the robot's links, actuators, and sensors. URDF is compatible with most of these simulators [7].

2.1.1 URDF

In 2009, the developers of the Robot Operating System (ROS) introduced the Unified Robot Description Format (URDF), a standardised, standalone specification that uses XML to represent the kinematic and dynamic properties of a robot. In a URDF model, rigid bodies are defined by `<link>` tags. Their relative motion is constrained by `<joint>` tags.

Every `<link>` element typically contains three distinct nested blocks:

- **Visual (`<visual>`):** Defines the 3D geometry of the link using primitives or CAD meshes, such as COLLADA (`.dae`) and STL files. It also defines visual properties such as colour and materials for rendering.
- **Collision (`<collision>`):** Defines the simplified physical boundaries used by the simulator's collision-detection algorithms to resolve contact forces.
- **Inertial (`<inertial>`):** Specifies the link's mass, centre of mass, and 3×3 inertia tensor. These properties are required by the simulator's rigid-body physics engine to calculate acceleration according to Newton's laws of motion.

Similarly, `<joint>` tags connect a parent link to a child link (see Figure 2.1), defining the joint type (e.g., continuous, revolute, prismatic, or fixed) and parameters such as limits, joint axis vectors, and mechanical dynamics.

To represent complex 3D geometries, both the `<visual>` and `<collision>` tags can import external Computer-Aided Design (CAD) mesh files. The two most prevalent mesh formats supported by ROS and Gazebo are STereoLithography (`.stl`) and COLLADA (`.dae`) [7], [8], [9]. COLLADA files are highly recommended for the visual representation (`<visual>`) because they support rich colours, textures, and materials, whereas raw `.stl` files only represent simple 3D surface geometries without any colour or texture information.

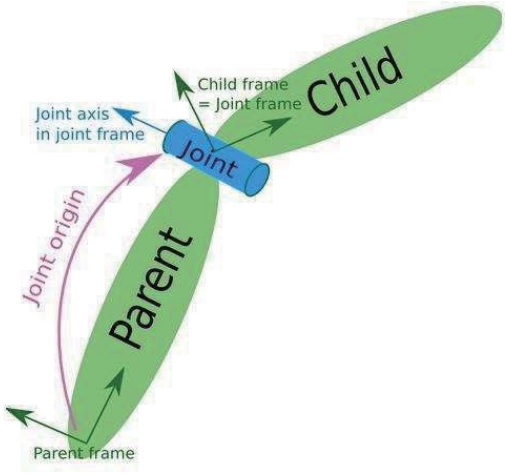


FIGURE 2.1: Link-Joint Coordinate Frames [8].

2.1.2 SDF

Due to limitations of the URDF format, the Simulation Description Format (SDF) was developed [10]. When a URDF-based robot is spawned in Gazebo, the simulator automatically parses the XML document and converts it to SDF. Custom `<gazebo>` extension tags can be added to the URDF to define simulation parameters. These include the surface friction coefficients μ_1 and μ_2, damping, contact stiffness, and ROS sensor plugins.

2.1.3 XML Macros (Xacro)

Writing URDF files in plain XML can quickly result in redundant and error-prone code. For this reason, the ROS ecosystem uses Xacro (XML Macros) as an XML-based preprocessor. It allows a geometric structure, such as a wheel link and joint, to be defined once and instantiated several times with different arguments. Xacro also supports basic arithmetic calculations within XML attributes. For example, joint origins can be calculated from link dimensions, which reduces rounding errors caused by manual entry.

2.2 ROS and Gazebo

ROS and Gazebo provide a software-in-the-loop framework for safely developing and validating autonomous robots, with ROS handling communication and control and Gazebo simulating the robot and its environment [8], [10], [11].

2.2.1 Robot Operating System

The Robot Operating System (ROS) is an open-source, distributed software framework that provides a structured communication layer above host operating systems [12]. ROS breaks down complex robotic functionality into a collection of small, independent programs known as nodes.

These nodes run as separate processes and exchange data using three primary communication mechanisms:

- **Topics (`sensor_msgs`, `geometry_msgs`):** Asynchronous data streams based on a publish-subscribe model. Nodes interested in specific telemetry, such as

laser scans (`sensor_msgs/LaserScan`) or camera images (`sensor_msgs/Image`), subscribe to named topics, which are published by sensor driver nodes at a regular frequency [13].

- **Services:** Synchronous, request-response communication patterns designed for quick, occasional transactions, such as toggling a state or querying a sensor threshold.
- **Actions:** Asynchronous, goal-oriented protocols designed for long-running, non-blocking tasks. Actions use a higher-level protocol composed of topics to send goals, cancel requests, and provide continuous progress feedback to the client node.

By abstracting hardware interfaces and sensor data into standardised message structures (such as `geometry_msgs/Twist` for movement commands), ROS decouples the high-level autonomous navigation and decision-making logic from the physical actuators and sensors [7], [13].

2.2.2 Gazebo

While ROS manages the control and messaging logic, Gazebo provides the 3D physical environment required to test these algorithms. Gazebo is a multi-robot simulator that simulates rigid-body dynamics, joint constraints, and complex environmental physics in a three-dimensional workspace. To be more precise, Gazebo integrates third-party physics engines to compute plausible physical forces including mass, gravity, surface friction, and contact collisions at discrete time steps [1].

Additionally, Gazebo generates realistic sensor feedback matching the behaviour of physical hardware. It replicates visual rendering using OGRE-based graphics, streaming simulated camera images, 3D depth point clouds, and LiDAR ray-casts directly into the simulation space. This sensor realism is crucial for training perception-based algorithms, such as local obstacle avoidance and path planning [5].

2.2.3 The ROS-Gazebo Communication Loop

The integration between ROS and Gazebo is facilitated by specialised simulator plugins (such as `libgazebo_ros_diff_drive.so` or `libgazebo_ros_camera.so`) that run natively inside Gazebo's step-wise simulation loop [9], [14]. These plugins act as a real-time bilateral data bridge between the virtual world and the ROS communication network:

1. **Actuator Control:** The control plugin subscribes to a command velocity topic (typically `/cmd_vel` carrying a `geometry_msgs/Twist` message) published by the ROS navigation node. It decodes the commanded linear and angular velocities, translates them into individual wheel velocities based on the robot's physical wheel diameter and separation, and applies the corresponding torque forces to the joint models inside the physics engine.
2. **Telemetry and Sensor Feedback:** As the robot moves, Gazebo's physics engine calculates its new spatial state and sensor outputs. The sensor plugins capture this virtual telemetry (e.g., wheel rotation encoder steps, camera frames, and laser scans) and publish it back onto ROS topics (such as `/odom`, `/joint_states`, and `/scan`).

2.3 Sensor and State Information

For autonomous navigation and local obstacle avoidance, a mobile robot must maintain continuous awareness of its internal state and its spatial relationship to the external environment. In the ROS ecosystem, state estimates are standardised using the `nav_msgs/Odometry` interface published on the `/odom` topic. This high-dimensional message carries two essential components:

- **Pose (`geometry_msgs/PoseWithCovariance`):** Represents the robot's estimated 3D position (x, y, z) and orientation (expressed as a quaternion) relative to its start-up origin.
- **Twist (`geometry_msgs/TwistWithCovariance`):** Specifies the robot's instantaneous linear and angular velocities.

ROS uses the `tf/tf2` library to maintain the spatial relationships between the coordinate frames of the robot, its sensors, and the surrounding environment. These relationships are organised as a transform tree.

Standard ROS spatial frames (REP 103) adhere to the right-handed coordinate convention, where the positive X-axis points forward, Y points left, and Z points upwards. However, standard computer vision and 2D image projection packages (such as OpenCV) employ a rotated visual convention, where the X-axis points from left to right, Y points downwards, and Z points straight forward into the image plane.

For this reason, ROS camera models commonly define a standard camera frame and a separate optical frame connected by a fixed transformation, which are illustrated in the URDF description:

1. **Standard Camera Frame (`camera_link`):** Placed at the physical mount location of the sensor, aligned with the X-forward ROS standard to ensure correct collision and TF tree parsing.
2. **Optical Frame (`camera_link_optical`):** The optical frame has the same origin as `camera_link`, but its axes are rotated to follow the computer-vision convention. In a typical URDF configuration, this transformation is represented by a roll of -$\pi/2$, a pitch of -$\pi/2$, and zero yaw.

For a robot to navigate dynamically in unknown environments, relative odometry must be supplemented by active perception sensors that capture external spatial data.

Depth Cameras and Point Clouds Depth cameras are active visual sensors that resolve the distance to surrounding obstacles by projecting an infrared structured light pattern onto surfaces or measuring the time-of-flight (ToF) of emitted light pulses. These sensors capture three-dimensional geometric structures that are invariant to environmental illumination changes or shadows [3].

The Gazebo depth camera plugin processes these distance measurements and publishes them as an unstructured point cloud on the `/camera/depth/points` topic using the `sensor_msgs/PointCloud2` message type. Each point in this cloud contains the exact (X, Y, Z) spatial coordinates of a detected surface relative to the sensor origin. This point cloud data provides the three-dimensional geometric foundation required for the Reinforcement Learning agent to calculate distance states and execute local obstacle-avoidance commands [5], [13].

2.4 Navigation and Obstacle Avoidance

The capability of a robot to successfully travel to the destination point is governed by the principles of robotic navigation and local obstacle avoidance. While closely related, these two tasks operate on different spatial and temporal scales: global navigation focuses on long-term path planning using prior environmental maps, whereas local obstacle avoidance is a reactive, real-time process that modifies the robot's immediate trajectory to prevent collisions with unforeseen or dynamic hazards [13], [15].

Robot motion can be controlled either manually through teleoperation or autonomously.

- **Manual Control (Teleoperation):** Under manual control, a human operator directly controls the robot, using keyboards, joysticks, or virtual interfaces. However, it requires continuous attention and communication, and fails to scale to multi-robot deployments or remote operations where signal latency is high [3], [16].
- **Autonomous Control:** In contrast, autonomous control enables a robot to perform goal-directed tasks without human intervention through a continuous sense-plan-act cycle. It relies on on-board perception sensors (such as LiDAR or depth cameras) that capture raw environmental data, planning algorithms that compute collision-free trajectories, and controllers that execute them despite sensor noise and dynamic conditions [17].

Traditional autonomous navigation systems rely on rule-based, analytical methods to perform local obstacle avoidance. Common classical approaches include Artificial Potential Fields (APF), which treat the robot as a point charge attracted to the goal and repelled by obstacles, and the Dynamic Window Approach (DWA), which evaluates dynamically feasible velocity search spaces [5], [18].

Classical obstacle-avoidance methods are efficient but can become trapped in local minima and often require extensive manual tuning. These limitations motivate data-driven reinforcement learning approaches, such as Q-learning, which allow robots to learn navigation policies through interaction with the environment.

2.5 Reinforcement Learning

Reinforcement Learning (RL) enables an agent to learn goal-directed behaviour through trial-and-error interaction with its environment, using rewards and penalties [4], [18].

The interaction between the robot (the agent) and the virtual or physical world (the environment) is formalised as a Markov Decision Process (MDP). It is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. Here, $\mathcal{S}$ is the state space and $\mathcal{A}$ is the action space. The transition function is denoted by P, the reward function by R, and the discount factor by $\gamma \in [0, 1)$. The agent aims to identify a policy π that achieves the highest expected sum of discounted rewards.

Within this MDP framework, Q-learning estimates the optimal action-value function $Q^*(s, a)$. It gives the expected return when the agent first takes action $a \in \mathcal{A}$ in state $s \in \mathcal{S}$ and follows an optimal policy thereafter. Q-learning is model-free because it learns directly from experienced transitions without knowing P. It is also off-policy. The agent can act using an exploratory behaviour policy, such as ε-greedy, while the update uses the greedy next-action value $\max_{a \in \mathcal{A}} Q(S_{t+1}, a)$ as its target.

Under sufficient exploration and appropriate learning-rate conditions, the estimate can converge towards Q^* even when the actions taken during training do not always follow the greedy policy. At each discrete time step t, Q is updated according to:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a \in \mathcal{A}} Q(S_{t+1}, a) - Q(S_t, A_t) \right] \tag{2.1}$$

Here, $\alpha \in (0, 1]$ is the learning rate, controlling how strongly a new estimate updates $Q(S_t, A_t)$. The discount factor $\gamma \in [0, 1)$ determines how much future rewards contribute to the return relative to immediate rewards. A third parameter, the exploration rate ε, does not appear in the update rule itself but governs how actions are chosen during training. The agent selects an action at random with probability ε and follows the greedy choice otherwise. During training, ε is typically reduced gradually toward a small floor value so that the agent increasingly favours exploitation over exploration.

When $\mathcal{S}$ and $\mathcal{A}$ are small and discrete, Q can be stored explicitly as a table indexed by state–action pairs (tabular Q-learning). For larger or continuous spaces, it is instead approximated by a function such as a neural network, as in deep Q-networks [3], [5].

Training proceeds episodically. Before the first episode, every entry $Q(s, a)$ is initialised together with the parameters α, γ, and ε. At the start of each episode, the agent is in the initial state S_0. At step t, the agent selects action A_t in state S_t using a policy derived from the current Q-values. The policy is typically ε-greedy. The agent executes A_t and observes reward R_{t+1} and the next state S_{t+1}. The entry $Q(S_t, A_t)$ is then updated using Equation 2.1. The process continues from S_{t+1} until a terminal state is reached.

For the specific task of local obstacle avoidance, the reward function is constructed to guide the agent's behaviour. It commonly combines positive rewards for progress towards a destination, a large reward for reaching the goal, and severe penalties for collisions [3], [5]. Simpler variants omit the goal term and instead reward collision-free forward and turning actions, as used later in this thesis (Chapter 5). Through repeated training episodes, the agent seeks an optimal policy π^* that yields the highest expected sum of discounted rewards.

Chapter 3

Related Work

Obstacle avoidance is one of the most relevant applications of RL in mobile robotics. Conventional navigation methods, such as A*, Dijkstra's algorithm and artificial potential fields, rely on an explicit representation of the environment and predefined rules for selecting collision-free paths. Although effective in structured, known environments, they require repeated path recalculation when previously unknown obstacles are detected, and potential-field methods can become trapped in local minima. RL allows the robot to learn an avoidance policy directly through interaction with its environment. Instead of programming an explicit response for every obstacle configuration, the robot observes its surroundings, selects a motion command and receives a reward according to the outcome of that action. This chapter first reviews RL approaches to mobile-robot obstacle avoidance and related navigation tasks, the application area most closely related to this thesis, and then briefly surveys RL applied to manipulation, motor-skill sequencing and legged locomotion, in order to place the present work in a wider context.

Deep Reinforcement Learning for Mobile-Robot Obstacle Avoidance Nguyen et al. presented a vision-based obstacle-avoidance method for a mobile robot equipped with a stereo camera [5]. Captured images were processed with YOLOv8 to detect obstacles, while depth information was used to estimate their distances. The image was divided into five horizontal regions, and the shortest obstacle distance within each region, together with the distance and heading angle to the target, formed the robot's state. A Deep Q-Network was trained to select from a set of predefined linear- and angular-velocity commands, using a reward that encouraged movement towards the goal and penalised collisions. Simulations in ROS 2 and Gazebo showed that the robot could reach a target while avoiding obstacles. The DQN therefore operated on a compact state extracted from the visual observations rather than on the full images.

İnner and Chachoua investigated the role of visual input in their *Depth Matters* study [19]. Within a goal-guided Transformer and Soft Actor-Critic framework, they compared RGB-D fusion with stacked RGB frames and other sensory configurations. Their simulation experiments found that early fusion of aligned colour and depth improved goal-reaching performance over the RGB baseline under the tested settings. Combining colour and depth gave the policy access to both appearance and obstacle-distance information.

Dooraki and Lee proposed a memory-based deep reinforcement learning (MDRL) framework for autonomous exploration and obstacle avoidance in unknown environments [3]. Depth-camera and range-sensor measurements were fused into the robot's state to cover the camera's near-range blind spot (< 50 cm), and an ϵ-greedy policy balanced exploration and exploitation while a frame-stacked short-term memory component allowed the network to distinguish perceptually similar locations and prevent corner-trapping loops. Unlike approaches relying on a pretrained model, the

agent learned entirely from scratch and kept adapting as the environment changed. The framework was evaluated in Gazebo and on a physical Kobuki robot.

Yang et al. introduced spatially-enhanced recurrent units for long-range mapless navigation using depth observations from a forward-facing stereo camera [20]. Their attention-based recurrent policy integrates observations across changing viewpoints to support spatial memory. Training included pretraining on synthetic depth data, and the system was deployed on a physical wheeled-legged robot without policy fine-tuning.

Shen et al. proposed ACR-Nav for corridor navigation using sectorised LiDAR measurements, scalar distance to a goal, and short histories of range measurements and executed actions [21]. A neural policy was trained using Proximal Policy Optimization (PPO), while a separate LiDAR-based safety filter activated near obstacles. Evaluation covered static and moving obstacles in simulated corridors. Its results concern the combined policy and safety filter rather than the learned policy alone.

Xin et al. developed a goal-guided dual-policy DQN framework for multi-robot navigation [22]. The network selects discrete linear velocities, while a PID controller determines angular velocity towards subgoals along a global reference path. Separate policies address individual motion and interactions between robots. Simulation and physical-robot experiments evaluated navigation efficiency and collision avoidance. Assigning angular control to the PID controller reduced the learned action space to discrete linear speeds.

Himaruwan et al. applied PPO to exploration and chemical-spill coverage in unknown environments under a limited energy budget [23]. The agent uses a partially observed grid map derived from simulated LiDAR and spill detections. Its reward encourages both exploration of unknown areas and cleaning of detected spills within the energy budget. Evaluation used a grid-world simulation with idealised sensing and without continuous robot dynamics.

Vu and Liu applied PPO to selecting a mobile robot's accompanying position relative to a person [24]. Their system combines the learned strategy with model predictive path integral (MPPI) control and control barrier functions for tracking and obstacle avoidance. Indoor and outdoor physical experiments demonstrated adaptive accompaniment. The learned policy selects an interaction strategy, and the tracking controller converts it into motion commands.

Manipulation and Grasping with Deep Reinforcement Learning Robotic manipulation is another major application area of RL. Levine et al. used Guided Policy Search to train deep visuomotor policies [25]. These policies mapped raw camera pixels directly to joint torques. They were demonstrated on a PR2 robot performing tasks such as peg insertion and placing shapes into a sorting cube.

More recently, Tang et al. used PPO for target approach and end-effector alignment of a FAIRINO FR3 robotic arm in PyBullet [26]. The policy used ten geometric values to describe the state. The reward encouraged the arm to move closer to the target and align its end effector, while penalising it for hovering over other objects. The study evaluated alignment in simulation rather than a complete physical grasping task. The policy learned to approach and align with the target while reducing misalignment with distractors. Han et al. provide a broader survey of DRL algorithms and challenges for robotic manipulation [27].

Grasping has also been scaled through self-supervised data collection. Kalashnikov et al. developed QT-Opt, a continuous-action Q-learning method trained on more than 580,000 real-world grasp attempts from seven robots [28]. It discovered emergent behaviours such as pre-grasp object singulation.

To limit the exploration required for precise contact tasks, Johannink et al. combined a hand-designed impedance controller with a learned residual RL policy [29]. They applied this approach to block assembly. Chebotar et al. combined model-based and model-free policy updates [30]. Their method learned tasks such as power-plug insertion within roughly an hour of real-world experience.

More recently, Luo et al. combined vision-based RL with human demonstrations and interventions during training [31]. Their system learned physical manipulation tasks involving precision assembly, dynamic manipulation, and dual-arm coordination, with reported training times of approximately one to two and a half hours.

Motor-Skill Learning and Sequencing RL has also been used to acquire and sequence individual motor skills. Peters and Schaal represented movements as parameterised dynamic motor primitives, using an episodic natural actor-critic algorithm to train a robot arm to hit a baseball by optimising a low-dimensional set of movement parameters rather than every motor command independently [32].

Zhang et al. combined implicit behaviour cloning with dynamic movement primitives and off-policy RL for robot motion planning [33]. Human demonstrations guided learning, while the movement primitives provided a simpler space for planning motions. The authors reported faster training and higher task scores than the compared RL baselines, and evaluated the method on a simple assembly task using a physical robot.

Beyond learning individual movements, complex tasks often require several skills in sequence. Alves et al. combined reinforcement learning with curriculum learning to acquire and connect skills for long-horizon robot manipulation tasks [34]. Their method extracts individual skills from expert demonstrations and uses their demonstrated order to learn transition policies between them. The curriculum guides training within each subtask, while the transition policies connect the learned skills into a complete sequence. The authors reported solving a robotic manipulation task using a single demonstration and reusing previously learned skills without additional training.

Legged Locomotion and Obstacle Negotiation RL has also been applied to obstacle negotiation on legged platforms. In this case, avoidance requires coordinated control of body posture, balance, and contact locations rather than steering alone. Tan et al. used domain randomisation over mass, friction, and surface properties for sim-to-real transfer [35]. They trained stable Minitaur walking gaits entirely in simulation before zero-shot hardware deployment.

Hoeller et al. trained walking, jumping, climbing, and crouching skills for the ANYmal quadruped using RL [36]. A high-level navigation policy selected and controlled these skills to cross successive obstacles, while a learned perception module reconstructed the surrounding geometry from noisy and partially occluded sensor observations. The system was trained using simulated data and transferred to a physical robot, which completed obstacle courses involving gaps, elevated surfaces, and low clearances.

Overall, RL has been applied to a broad range of robotic problems, from mobile-robot obstacle avoidance to legged locomotion, manipulation, grasping and motor-skill sequencing. The reviewed navigation studies differ in their observations, use of memory, task objectives, and integration with conventional controllers. For the mobile robot considered in this thesis, five discretised depth sectors and three motion commands make tabular Q-learning a practical alternative to a neural policy. The

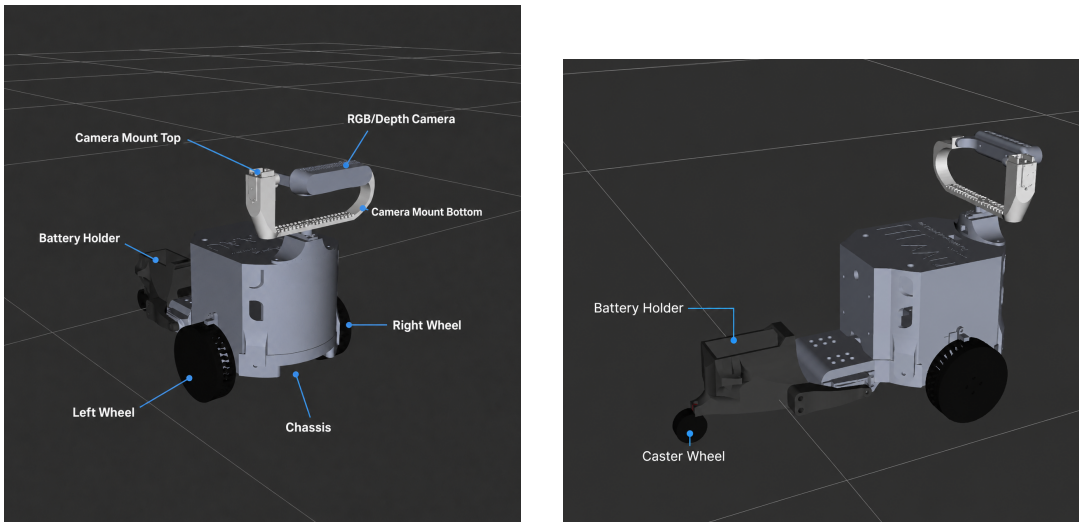
task concerns local obstacle avoidance without a target position. Its state–action values can be inspected directly, but the compact representation discards geometric detail and observation history. The evaluation therefore examines the capabilities and limitations of this formulation in previously unseen room layouts.

Chapter 4

System Setup and Implementation

4.1 Software and Hardware Overview

At the outset of the project, a preliminary CAD model of the Pio robot was provided as the basis for the simulation model. Its geometry was supplied as STL mesh files representing the `Chassis`, `Battery_Holder`, `Wheel_Left`, `Wheel_Right`, `Camera_MountBottom`, and `Camera_MountTop` components. Since STL files contain no colour information, the colours of the individual components were assigned manually according to the preferred visual appearance. Figure 4.1 shows front and side views of the resulting model.



(A) Front view (B) Side view

FIGURE 4.1: Front and side views of the Pio robot model and its main components

An initial URDF description was also provided. It defined the links and joints connecting the components, together with their visual geometry, collision geometry, mass, and inertial properties. However, the model was not yet suitable for reliable dynamic simulation in Gazebo and required corrections to its structure, joint configuration, coordinate transformations, and physical parameters. Figure 4.1 shows the robot after the modification, with an additional passive rear caster wheel added as a third ground-contact point to improve stability. Further details are provided in Section 4.3.

The resulting simulated platform uses a differential-drive configuration with two independently driven wheels and a passive rear caster. An Intel RealSense D435 camera is mounted above the chassis and provides RGB and depth data for

perception, although the obstacle-avoidance controller developed in this work uses only the depth measurements.

The software environment was based on Ubuntu 20.04 and ROS Noetic and was organised as a ROS 1 Catkin package. Xacro and URDF describe the robot's structure and physical properties, Gazebo simulates its dynamics, wheel actuation, sensors, and interaction with the environment, and `gazebo_ros` provides the interface between the simulator and ROS. RViz is used to inspect the robot model, coordinate frames, and camera output.

4.2 Robot Model Preparation

The robot description was organised using Xacro to separate the main robot structure from Gazebo-specific configuration. Xacro's inclusion mechanism, implemented through `<xacro:include>`, allows external XML files to be combined into a single robot description.

The structure of the `pio_description` ROS package is shown in Figure 4.2. The package contains the robot meshes, URDF and Xacro descriptions, Gazebo world files, launch files, and package configuration files.

```
src/
└── pio_description/
    └── Asem_Reduced.SLDASM/
        ├── config/
        │   └── pio.rviz
        ├── launch/
        │   ├── display.launch
        │   └── gazebo.launch
        ├── meshes/
        ├── urdf/
        │   ├── Asem_Reduced.SLDASM.csv
        │   ├── Asem_Reduced.SLDASM.gazebo.xacro
        │   └── Asem_Reduced.SLDASM.urdf.xacro
        ├── worlds/
        ├── CMakeLists.txt
        └── package.xml
```

FIGURE 4.2: `pio_description` ROS package.

The main files have the following functions:

- **Robot description:** `Asem_Reduced.SLDASM.urdf.xacro` defines the main links, joints, visual meshes, collision geometry, and physical structure of the robot.
- **Gazebo configuration:** `Asem_Reduced.SLDASM.gazebo.xacro` contains Gazebo-specific properties and plugins, including the differential-drive controller and simulated camera configuration.
- **Launch files:** `gazebo.launch` starts the robot in the selected Gazebo environment, while `display.launch` opens the predefined `pio.rviz` configuration for model inspection.
- **RViz configuration:** `pio.rviz` sets `Chassis` as the fixed reference frame and displays a reference grid. It also displays the complete robot model from

the `robot_description` parameter and the simulated camera stream from `/camera/image_raw`.

- **Mesh files:** The `meshes` directory contains the STL models used to represent the visual geometry of the individual robot components. Additional mesh files can be added when extending the robot model.
- **World files:** The `worlds` directory contains five rooms for training and ten rooms for testing. Other Gazebo worlds can also be added, for example `turtlebot3_house.world`.

During the first simulation tests, the robot repeatedly tilted and fell onto its side. The initial instability was traced primarily to the configuration of the battery holder and the transformations inherited from the exported model.

One source of incorrect behaviour was the difference between the coordinate systems used by the exported SolidWorks meshes and those expected in the URDF model. STL files contain only geometric information, while their positions and orientations within the robot are defined separately by the `origin` elements of the URDF. Incorrect mesh origins or inconsistent axis orientations can therefore cause components to appear displaced, rotated, or incorrectly aligned. The corresponding link and joint transformations were inspected and adjusted to reconstruct the intended assembly in Gazebo.

The `Battery_Holder` joint was also corrected. In the originally exported model, it was defined as a `revolute` joint, which allowed the battery holder to swing relative to the chassis and caused the initial instability. It was therefore changed to a `fixed` joint. Contact parameters were also assigned to the link ($\mu_1 = 0.05$, $\mu_2 = 0.05$, $k_p = 1000000$, $k_d = 10$, minimum penetration depth 0.001 m, and maximum contact velocity 1.0 m/s), analogous to the tuning applied to the rear caster wheel described in Section 4.3.

After examining the complete assembly and the available STL files, a caster wheel was identified beneath the battery holder (see Figure 4.3b). This component was missing from the original URDF description and was therefore added to the simulated model. The caster wheel provides an additional ground-contact point and supports the rear part of the robot.

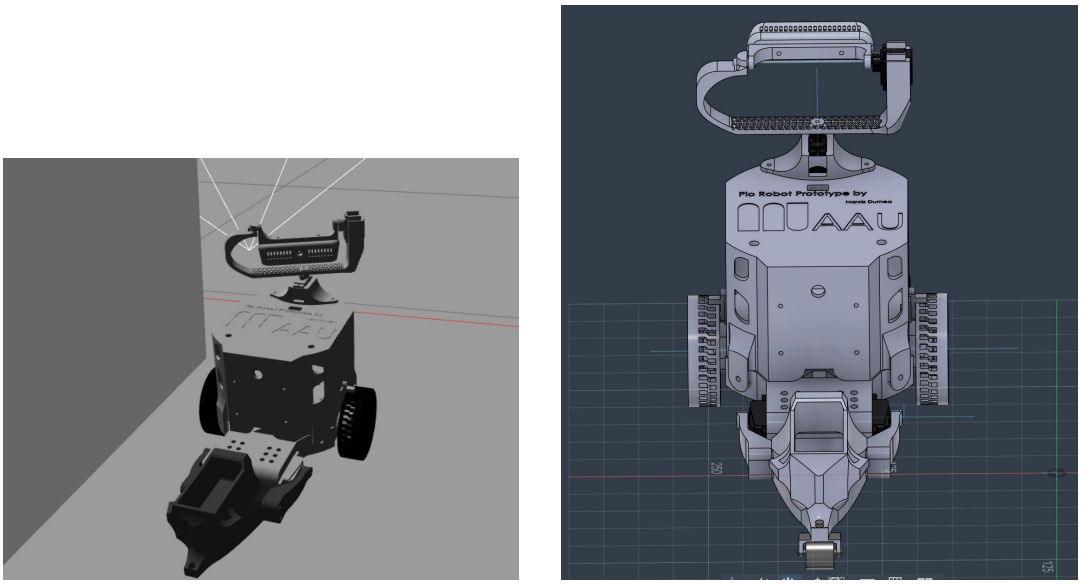
4.3 Rear Caster Wheel Implementation

The rear caster was implemented as a passive two-degree-of-freedom mechanism composed of the links `RearCasterFork` and `RearCasterWheel`. Unlike the two driven wheels, the caster is not connected to the differential-drive plugin and therefore receives no actuation command.

The fork is attached to the `Chassis` link through the `RearCasterSwivel_joint`, a continuous joint rotating about the vertical (z) axis, positioned at

$$x = 0.17\,\text{m}, \qquad y = 0.0725\,\text{m}, \qquad z = -0.005\,\text{m}$$

relative to the chassis frame, directly behind the battery holder. The wheel itself is attached to the fork through the `RearCasterWheel_joint`, a second `continuous` joint rotating about the horizontal (x) axis, offset by $(0,\ 0.006,\ -0.0175)$ m relative to the swivel axis. This two-joint arrangement allows the fork to swivel freely about the vertical axis while the wheel rotates about its own axle, reproducing the behaviour of a real caster wheel.



(A) Rear view of the initial URDF model (B) Complete robot assembly

FIGURE 4.3: Comparison between the initial URDF model and the complete Fusion assembly used to identify the missing caster wheel

Both links use simple cylindrical collision and visual geometries and were assigned small mass and inertia values, since the caster contributes negligible structural load compared to the main chassis and driven wheels. In the Gazebo description, the `RearCasterWheel` link was additionally configured with low friction coefficients ($\mu_1 = 0.08$, $\mu_2 = 0.05$) so that it rolls with minimal resistance, together with contact stiffness and damping parameters ($k_p = 100000$, $k_d = 10$), a minimum penetration depth of 0.0005 m, and a maximum contact correction velocity of 0.1 m/s to keep the contact solver stable.

4.4 Drive and Motion Control

The drive system translates velocity commands received on the `cmd_vel` topic into wheel motion. This is realized in simulation through the `libgazebo_ros_diff_drive` Gazebo plugin, which is attached to the robot model and configured with the two driven wheel joints, converting the linear and angular velocity commands into the corresponding left and right wheel speeds. The same `cmd_vel` interface is used for both manual and later autonomous control, so no separate command path had to be implemented for the reinforcement-learning controller.

To verify that the robot moved as expected, it was manually driven using the keyboard teleoperation node from the TurtleBot3 package, which publishes velocity commands to `cmd_vel`. After starting the Gazebo simulation, the keyboard controller was launched in a separate terminal using:

```
roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

This test revealed that the left and right wheel joints were swapped in the plugin configuration: commands intended to turn the robot in one direction caused it to turn in the opposite direction. The joint assignment in the plugin configuration was corrected accordingly, after which the observed motion matched the commanded direction.

4.5 Odometry Setup

The same `libgazebo_ros_diff_drive` plugin also provides odometry, publishing it on the `/odom` topic and broadcasting the corresponding `odom`→`Chassis` transform. Rather than integrating simulated wheel motion, the plugin is configured to take the odometry directly from the robot's true pose in Gazebo (`odometrySource: world`), so no additional filtering or sensor fusion is applied. This odometry output was validated by comparing it with the robot's actual movement in Gazebo during manual and autonomous operation.

4.6 Camera Integration

In the original robot model, the camera was attached through a continuous rotational joint, which allowed pan and tilt motion but caused the camera to sag downward under gravity instead of maintaining a stable view of the area in front of the robot. Since this additional degree of freedom was not required for the obstacle-avoidance task considered in this project, the camera was instead fixed in a static orientation, mounted on top of the robot and directed forward. This simplified both the perception pipeline and the control problem, since the relationship between the camera observations and the robot's direction of motion remained constant.

The simulated Intel RealSense D435 camera provides both RGB and depth data. Although the RGB image stream remains available, only the depth data were used during reinforcement-learning training. The depth image directly shows the distance between the camera and visible surfaces, which is more suitable for detecting nearby obstacles and for its avoidance.

The Gazebo depth-camera plugin publishes the depth image on:

`/camera/depth/image_raw`

It also publishes the corresponding point cloud on:

`/camera/depth/points`

The RGB camera was configured with a resolution of 320×240 pixels, an update rate of 30 Hz, and a horizontal field of view of approximately 1.086 rad. The near and far clipping distances were set to 0.03 m and 10 m, respectively. The depth camera is configured with the same 320×240 resolution and has a maximum range of 5 m, while the point-cloud distance range is limited to approximately 0.03–5 m.

Figure 4.4 shows the RGB and depth images published by the simulated camera.

The robot's base frame is defined as `Chassis`. Based on the fixed transformations in the URDF, the position of `camera_link` relative to `Chassis` is approximately:

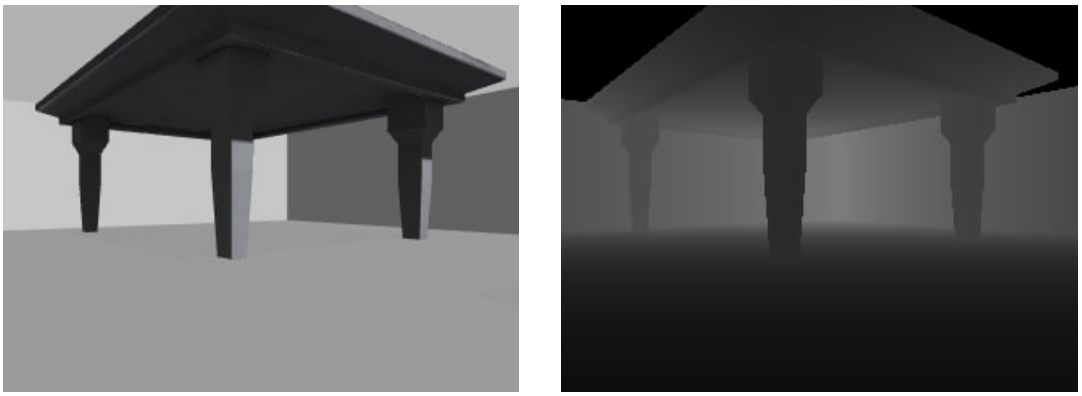
$$x = 0.17418\,\text{m}, \qquad y = -0.27145\,\text{m}, \qquad z = 0.20316\,\text{m}.$$

Thus, the camera origin is located approximately 0.203 m above the origin of the chassis frame. Its orientation relative to `Chassis` is approximately:

$$\text{roll} = 0, \qquad \text{pitch} = 0, \qquad \text{yaw} = -1.5708\,\text{rad}.$$

The camera joint itself contains the rotation

`rpy="-1.5708 -1.5708 0"`



(A) RGB image (B) Depth image

FIGURE 4.4: RGB and depth images

An additional fixed transformation converts `camera_link` into `camera_optical_frame`.

To verify the correctness of the camera configuration, an RViz configuration (`config/pio.rviz`) was created to visualize the robot model, RGB and depth camera streams, coordinate frames, and odometry data. The robot description and the relative placement of the camera frame can be inspected without running the physics simulation using:

```
roslaunch pio_description display.launch
```

The same configuration can also be opened alongside a running Gazebo simulation. This makes it possible to inspect the camera streams in real time and verify that the images are published correctly, that the camera is directed towards the area in front of the robot, and that its coordinate frames are properly aligned with the robot model.

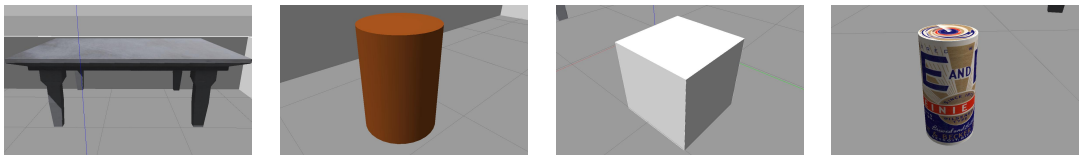
4.7 Simulation Environment

Several Gazebo environments were configured for the Pio robot, including an empty world, the TurtleBot3 house, and a collection of custom rooms. Five custom rooms were created for training, while ten distinct rooms were used for testing (see Figures 4.6, 4.7, respectively). Each room consists of a 5 m $\times$ 5 m enclosure bounded by four walls. The rooms vary in the number, type, and arrangement of obstacles, which include a marble table, cylindrical and cubic objects, and several medium-height cans (see Fig. 4.5). Obstacles of sufficient height were selected to ensure reliable detection by the robot's depth camera, since objects positioned below the camera's field of view may not be detected at close range. The empty world and TurtleBot3 house were imported from the TurtleBot3 simulation package.

The desired environment can be selected through the `world` argument of the `gazebo.launch` file. For example, the TurtleBot3 house can be launched using:

```
roslaunch pio_description gazebo.launch world:=turtlebot3_house
```

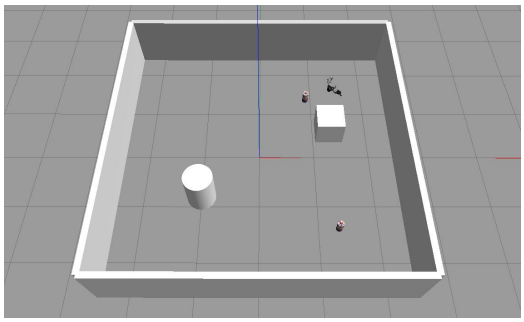
The `gazebo.launch` file initializes the complete Pio simulation. First, the robot description is generated from the Xacro model and loaded onto the ROS parameter server as `robot_description`. The launch file then loads the selected Gazebo world



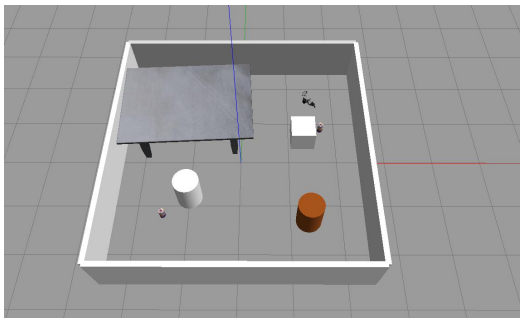
(A) Marble Table (B) Cylinder (C) Box (D) Medium size can

FIGURE 4.5: Obstacles present in the environment

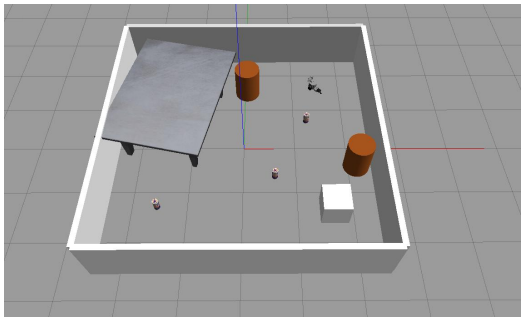
and spawns the Pio model at a predefined position and orientation. Finally, the `robot_state_publisher` node publishes the transformations between the robot's coordinate frames.



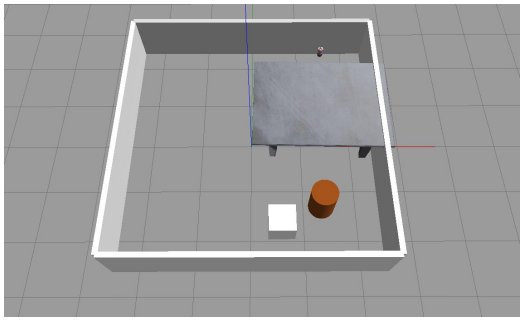
(A) Room 1



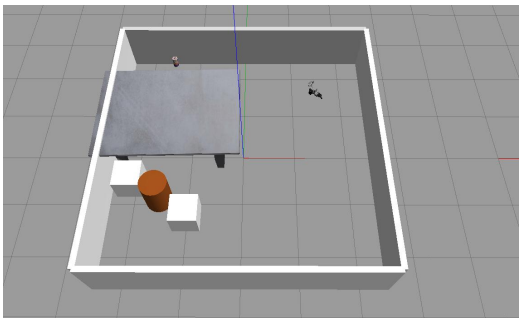
(B) Room 2



(C) Room 3

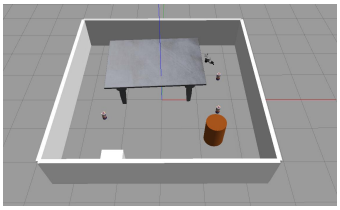


(D) Room 4

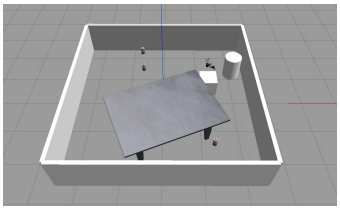


(E) Room 5

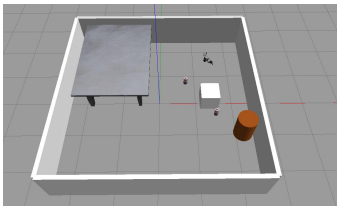
FIGURE 4.6: Custom rooms used for training



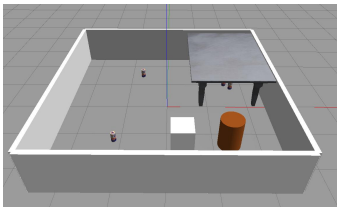
(A) Room 1



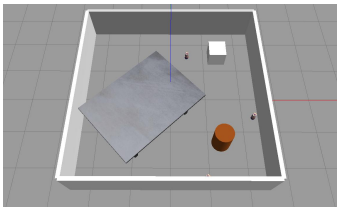
(B) Room 2



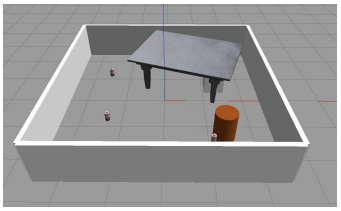
(C) Room 3



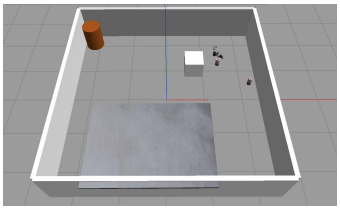
(D) Room 4



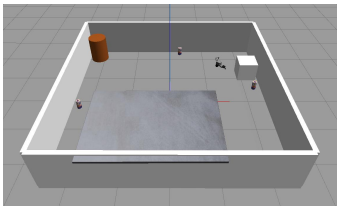
(E) Room 5



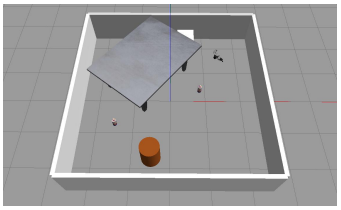
(F) Room 6



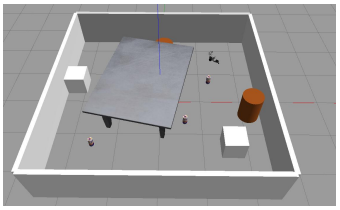
(G) Room 7



(H) Room 8



(I) Room 9



(J) Room 10

FIGURE 4.7: Custom rooms used for testing and evaluating

Chapter 5

Reinforcement Learning Approach

For training the Pio robot to navigate the custom Gazebo room and avoid obstacles, a tabular Q-learning approach is used. At every control step the agent (the robot) observes a state derived from the depth camera, selects one of three discrete actions, and receives a scalar reward that depends on the chosen action and on whether the step ends the episode. Over many episodes, the agent updates the Q-table and gradually improves its policy. Tabular Q-learning was chosen over other reinforcement learning algorithms for its simplicity, low computational cost, and ease of debugging. Training and evaluation were carried out in the custom room environments described in Section 4.7.

The environment is registered with `openai_ros` as the task-and-robot environment `PioDepthObstacle-v0`, implemented by the `PioDepthObstacleEnv` class, which builds on a robot-specific `PioEnv` superclass. `PioEnv` subscribes to `/odom` and `/camera/depth/image_raw` and exposes a `move_base` method that publishes the corresponding `cmd_vel` command for each discrete action. `PioDepthObstacleEnv` adds the logic: converting the raw depth image into a discrete state, computing the reward, and deciding when an episode ends.

5.1 Implementation Details

The RL-related part of the system uses sensor observations from the robot's depth camera to decide between three discrete actions. Special attention is given to the definition of the observation space, action space, and reward structure, described below.

5.1.1 Depth-Image Discretization

The raw input to the agent is the 320×240 depth image published on `/camera/depth/image_raw` (see Chapter 4). Because the Q-learning implementation (`qlearn.QLearn`) requires a discrete state, this image cannot be used directly as an observation and is instead reduced to a small vector of symbolic values through the following steps, performed in `discretize_depth_observation` function:

1. **Conversion:** the ROS `Image` message is converted to a NumPy array using `cv_bridge`, with `passthrough` encoding, and cast to `float32`.

2. **Unit normalisation:** depending on the driver, depth values may be reported in metres or millimetres. The median of the finite pixel values is checked. If it exceeds 20, the array is assumed to be in millimetres and is divided by 1000 to obtain metres.

3. **Handling of invalid readings:** depth cameras commonly report `NaN` or `Inf` for pixels with no valid return. These values are replaced with `max_depth_range` (the configured sensor range, 5.0 m), so that an invalid reading is treated as "no obstacle detected" rather than as a close obstacle. The array is then clipped to $[0,$ `max_depth_range`$]$.

4. **Cropping:** only the vertical band between 10% and 85% of the image height is kept, to exclude the floor and ceiling/upper background from the obstacle estimate and focus on the region in front of the robot at chassis height.

5. **Angular binning:** the cropped image is split horizontally into five equal-width vertical slices (far-left, centre-left, centre, centre-right, and far-right), each corresponding to one entry of the state vector.

6. **Robust distance estimate per slice:** within each slice, the pixels are further split into eight column strips. For each strip, the 2nd percentile of the closest surface in that strip is taken. That reduces the influence of a small number of noisy or spurious pixels compared to a plain minimum. The smallest of these eight per-strip values is then used as the slice's distance. Slices with no valid pixels default to `max_depth_range`.

7. **Discretization into state symbols:** each slice distance is finally mapped to one of three discrete levels using two configurable thresholds, `min_range` (0.20 m) and `medium_range` (0.65 m):

$$\text{level} = \begin{cases} 0 & \text{if slice distance} < \texttt{min_range} \\ 1 & \text{if } \texttt{min_range} \leq \text{slice distance} < \texttt{medium_range} \\ 2 & \text{otherwise} \end{cases}$$

   where 0 is obstacle in the critical range, 1 is obstacle in medium distance and 2 is clear up to `max_depth_range`.

The result is a 5-tuple of integers, each in $\{0, 1, 2\}$, one per vertical slice of the depth image. For example, the state $(2, 2, 2, 0, 0)$ indicates that the closest obstacle in the leftmost three slices is far away, and the closest obstacles in the rightmost two slices are in the critical range. The resulting state space consists of $3^5 = 243$ discrete states.

The depth camera additionally has a near clipping distance of 0.03 m. Surfaces closer than this limit are outside the camera's valid measurement range and are therefore not considered close obstacles by the discretization procedure. This lower limit mostly prevents parts of the robot located very close to the camera, such as components of the camera mount or robot body, from being interpreted as obstacles. It should not be confused with `min_range` (0.20 m), which defines the critical obstacle range and is used to terminate an episode.

Independently of the discretization, the environment also keeps track of the raw minimum distance, `closest_depth`, the smallest distance between 5 sectors. So, as soon as `closest_depth` falls below `min_range`, the episode ends.

5.1.2 Action Space

The action space consists of three discrete actions: move forward (0), turn left (1), and turn right (2). Table 5.1 lists the corresponding velocity commands sent to `cmd_vel`.

TABLE 5.1: Discrete action space used for training and evaluation.

Action	Linear speed (m/s)	Angular speed (rad/s)
Move forward	0.10	0.0
Turn left / right	0.02	0.3

5.1.3 Reward Function

The reward assigned at each step depends only on the action performed and on whether that action terminates the episode. The observed state does not otherwise directly affect the reward value. Let A_t denote the action executed at step t, and let $d_t^{\min}$ denote the smallest slice distance at that step (Section 5.1.1). Consistent with the notation in Chapter 2, the reward returned after executing A_t is denoted by R_{t+1}:

$$R_{t+1} = \begin{cases} -200 & \text{if } d_t^{\min} < 0.20\,\text{m (terminal)}, \\ +5.0 & \text{if } A_t = \text{FORWARD}, \\ +1.0 & \text{if } A_t \in \{\text{TURN_LEFT}, \text{TURN_RIGHT}\}. \end{cases} \tag{5.1}$$

The forward action receives a higher reward than the turning actions. This encourages the robot to move through the room instead of turning in place. A penalty is applied when an episode ends in a collision. Both turning actions receive the same reward. The reward function is therefore symmetric under a left–right relabelling. Unlike the goal-directed methods discussed in Chapter 3, this task has no target position. Its purpose is open-ended collision-free navigation rather than goal reaching.

5.1.4 Learning Algorithm

The agent is trained with the tabular Q-learning rule introduced in Chapter 2 (Equation 2.1). In the Pio implementation, one interaction with Gazebo produces the transition $(S_t, A_t, R_{t+1}, S_{t+1})$. The variables S_t and S_{t+1} are the five-symbol depth states from Section 5.1.1. The variable A_t is one of the three velocity commands listed in Table 5.1. Only the table entry for the observed pair (S_t, A_t) is updated:

$$Y_t = \begin{cases} R_{t+1} + \gamma \max_{a \in \mathcal{A}} Q(S_{t+1}, a) & \text{if } S_{t+1} \text{ is non-terminal}, \\ R_{t+1} & \text{if } S_{t+1} \text{ is terminal}, \end{cases} \tag{5.2}$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_e \left[Y_t - Q(S_t, A_t) \right]. \tag{5.3}$$

The expression inside the brackets is the difference between the previous estimate and the target Y_t obtained from the current transition. For a non-terminal step, the target combines the immediate reward with the largest value available in the next depth state. A collision terminates the episode. In this case, there is no subsequent action to evaluate. The target Y_t therefore consists only of the -200 collision reward.

Action selection during training is ε-greedy. A random action is selected with probability ε_e. Otherwise, an action with the largest Q-value in the current state is used. The implementation selects randomly among actions that share the largest value. Both the learning rate and exploration rate are reduced once per episode e according to

$$\begin{aligned} \alpha_e &= \max(0.01,\, 0.1(0.997)^e), \\ \varepsilon_e &= \max(0.1,\, 0.9(0.997)^e). \end{aligned} \tag{5.4}$$

Early episodes therefore make larger corrections and explore more often. Later episodes retain a small amount of learning and exploration. The discount factor remains fixed at $\gamma = 0.9$. Table 5.2 summarises the training parameters.

TABLE 5.2: Q-learning hyperparameters.

Parameter	Value
Initial learning rate α_0	0.1
Learning-rate decay (per episode)	$\times 0.997$
Minimum learning rate	0.01
Discount factor γ	0.9
Initial exploration rate ε_0	0.9
Exploration decay (per episode)	$\times 0.997$
Minimum exploration rate	0.1
Episodes	1000
Max. steps per episode	1000

5.2 Training Procedure

The final training run contains 1000 episodes. Its main characteristics are:

- **Training environments.** Five rooms with different obstacle configurations shown in Fig. 4.6 are used. Training switches to the next room after every block of ten episodes.
- **Starting position.** The simulation is reset before each episode. The robot is placed at a random position outside the exclusion zones defined around the obstacles.
- **Agent interaction.** At each step, the agent selects A_t with the ε-greedy policy. It applies the action and observes R_{t+1} and S_{t+1}. The corresponding Q-table entry is then updated.
- **Episode termination.** A collision terminates the episode. An episode also stops when it reaches the limit of 1000 steps.
- **Parameter decay.** The learning rate decreases from 0.1 to a minimum of 0.01. The exploration rate decreases from 0.9 to a minimum of 0.1. The decay factor is 0.997 per episode for both rates.
- **Logging and checkpoints.** The step log stores the state, action, reward, closest depth, and cumulative reward. The episode log stores the collision outcome, action counts, minimum depth, and number of visited states. The current Q-table is saved after each episode. A permanent checkpoint is stored every 50 episodes.

5.2.1 Checkpoint Selection

Training performance did not improve monotonically. A collision-free training episode is one that reaches the 1000-step limit. The following results show how the performance changed during the run:

- During episodes 1–100, none of the 100 episodes was collision-free. The mean episode length was 174.4 steps. The mean reward was 282.8.
- During episodes 801–900, 50 of the 100 episodes were collision-free. The mean episode length increased to 678.1 steps. The mean reward increased to 2606.4.
- During episodes 901–1000, the number of collision-free episodes fell to 26. The mean episode length fell to 495.4 steps. The mean reward fell to 1822.0.

Episode 900 was selected because the strongest late training block ended at this checkpoint. In the remainder of this thesis, it is referred to simply as the trained model. Figure 5.1 shows the collision-free rate for each block of 100 training episodes.

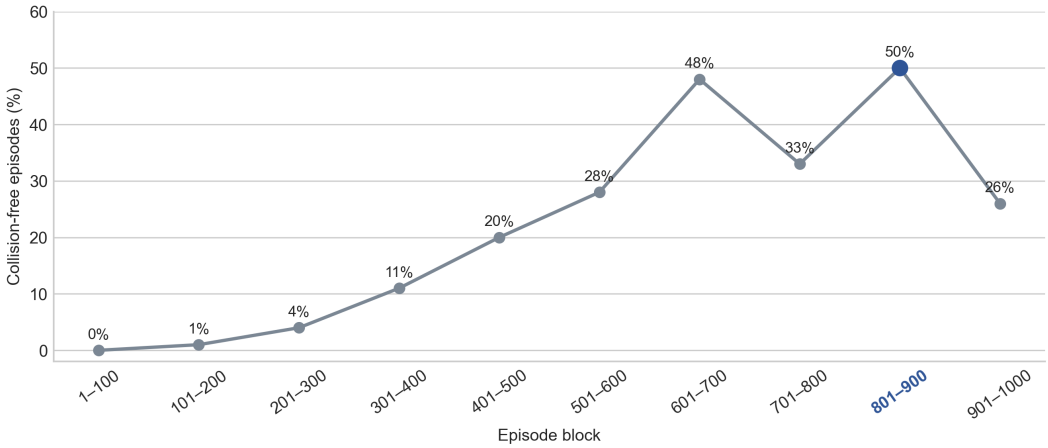


FIGURE 5.1: Collision-free training episodes per 100-episode block

5.2.2 Learned Policy

The theoretical observation space contains $3^5 = 243$ states. However, the symbol 0 indicates that an obstacle is already within the critical range. This observation normally terminates the episode before another action is selected. The relevant non-terminal states therefore contain only the symbols 1 and 2, giving $2^5 = 32$ possible states. At episode 900, the model had learned 30 of these states. This corresponds to a coverage of 93.75 % of the relevant state space.

Table 5.3 contains these 30 non-terminal states and one additional state containing 0. All three actions have learned values in 28 of the 30 non-terminal states. A dash marks an action that was not updated during training. Such an entry retains its initial value of zero during greedy action selection. Among the represented non-terminal states, the policy prefers forward in 10 states, left in 9 states, and right in 11 states.

TABLE 5.3: Learned Q-values for all represented states

State	Q(forward)	Q(turn left)	Q(turn right)	Best action
$(0, 1, 1, 2, 2)$	-5.71	-3.06	—	turn right
$(1, 1, 1, 1, 1)$	6.68	6.58	11.92	turn right
$(1, 1, 1, 1, 2)$	19.41	16.08	22.20	turn right
$(1, 1, 1, 2, 1)$	8.60	3.01	15.15	turn right

Continued on the next page

Table 5.3 continued

State	Q(forward)	Q(turn left)	Q(turn right)	Best action
$(1,1,1,2,2)$	26.03	22.62	27.99	turn right
$(1,1,2,1,1)$	17.10	10.23	11.68	forward
$(1,1,2,1,2)$	10.72	2.13	1.48	forward
$(1,1,2,2,1)$	32.22	22.34	20.63	forward
$(1,1,2,2,2)$	31.12	31.21	34.09	turn right
$(1,2,1,1,1)$	5.58	3.89	11.03	turn right
$(1,2,1,1,2)$	20.67	3.88	2.80	forward
$(1,2,1,2,2)$	6.45	3.47	19.10	turn right
$(1,2,2,1,1)$	10.85	14.34	10.54	turn left
$(1,2,2,1,2)$	1.56	0.35	5.76	turn right
$(1,2,2,2,1)$	20.43	26.25	20.30	turn left
$(1,2,2,2,2)$	42.38	38.59	39.55	forward
$(2,1,1,1,1)$	6.47	13.75	2.90	turn left
$(2,1,1,1,2)$	10.13	7.21	15.45	turn right
$(2,1,1,2,1)$	13.80	2.66	3.54	forward
$(2,1,1,2,2)$	17.01	13.70	22.61	turn right
$(2,1,2,1,1)$	0.40	—	0.24	forward
$(2,1,2,2,1)$	0.62	0.74	5.55	turn right
$(2,1,2,2,2)$	18.97	11.86	27.04	turn right
$(2,2,1,1,1)$	12.36	14.90	10.25	turn left
$(2,2,1,1,2)$	29.74	25.88	25.11	forward
$(2,2,1,2,1)$	—	2.99	—	turn left
$(2,2,1,2,2)$	11.76	22.44	5.42	turn left
$(2,2,2,1,1)$	23.04	24.04	23.16	turn left
$(2,2,2,1,2)$	30.23	11.07	11.50	forward
$(2,2,2,2,1)$	36.85	38.04	34.90	turn left
$(2,2,2,2,2)$	47.26	43.73	43.46	forward

Chapter 6

Experimental Evaluation and Discussion

6.1 Experimental Setup

The trained model was evaluated in the ten test rooms shown in Figure 4.7. These rooms were not used during training. The aim was to assess the model's obstacle-avoidance performance in previously unseen environments.

During testing, learning was disabled and the exploration rate ε was set to zero. This removed random exploratory actions and kept the Q-table unchanged. For each observed state, the robot selected the action with the highest Q-value. The resulting policy is deterministic, so the robot always chooses the same action for a given state.

Each room was evaluated in three trials, giving 30 trials in total. At the start of each trial, the robot received a random position and orientation. It then navigated for up to 300 s (5 min). A collision ended the trial immediately. A trial was considered successful if the robot completed the full duration without a collision.

6.2 Evaluation Metrics

The robot's performance depended on the room configuration and its random starting pose. It completed all three trials without a collision in Rooms 1, 3, and 6–10. In Rooms 2 and 5, collisions occurred in only some trials. This indicates that the starting pose also influenced the result. Room 4 was the most challenging configuration, as all three trials ended in a collision. Overall, the robot completed 24 of the 30 trials successfully. This corresponds to a collision-free rate of 80 %.

For each trial, the occurrence of a collision, the travelled distance, the number of steps, and the final cumulative reward were recorded. Table 6.1 presents the number of trials with a collision in each room. The remaining values are means across the three trials.

Table 6.2 summarises the results across all 30 trials. A fully successful room is one in which all three trials were collision-free. The action distribution was calculated from all 37,628 evaluation steps.

6.3 Qualitative Results

This section examines the performance in each test room in greater detail. It begins with Rooms 1, 3, and 6–10, where all three trials were successful. It then considers Rooms 2 and 5, where the outcome varied with the robot's starting pose. Finally, it analyses Room 4, where all three trials ended in a collision.

TABLE 6.1: Evaluation results for the ten test rooms.

Room	Collision trials	Distance (m)	Steps	Reward
1	0/3	25.49	1419.0	6081.7
2	2/3	19.53	1087.0	4354.3
3	0/3	24.84	1403.0	5839.0
4	3/3	8.72	497.0	1794.7
5	1/3	18.63	1045.7	4197.3
6	0/3	25.40	1418.7	6033.3
7	0/3	25.68	1419.3	6123.3
8	0/3	25.97	1414.3	5877.0
9	0/3	24.84	1418.7	5788.0
10	0/3	25.35	1420.0	6110.7
Overall	6/30	22.44	1254.3	5219.9

TABLE 6.2: Overall evaluation results and action distribution

Collision-free rate	Collision-free trials	Successful rooms	Forward	Left	Right
80 %	24/30	7/10	79.85 %	3.10 %	17.05 %

Figures 6.1, 6.2, and 6.3 show the room layouts and exact trajectories obtained from the logged (x, y) positions. The plotting script was developed with assistance from Claude (Anthropic).

6.3.1 Successful Trials

Every room except Room 4 had at least one collision-free trial. Figure 6.1 presents one successful trajectory from each of these nine rooms. Rooms 1, 8, and 10 are shown twice to demonstrate that different starting poses can produce successful routes with turns in both directions. However, the overall action distribution still favoured right turns. All twelve trajectories completed the full 300 s evaluation period without a collision.

6.3.2 Occasional Collisions

Rooms 2 and 5 produced both successful and unsuccessful trials. The three unsuccessful trajectories are shown in Figure 6.2. In Room 2, Trials 2 and 3 ended in a collision after approximately 223.4 s and 173.1 s. In Room 5, Trial 3 ended after approximately 63.2 s. The other trials in these rooms completed the full evaluation period. This variation indicates that the outcome depended on the random starting pose as well as the room configuration.

6.3.3 Consistent Failure in Room 4

Room 4 was the only configuration in which all three trials ended in a collision. Figure 6.3 shows the complete set of trajectories. Trials 1, 2, and 3 ended after approximately 134.7 s, 110.4 s, and 71.9 s, respectively. Trials 1 and 2 reached almost the same collision area, despite their different starting poses. Trial 3 collided at another location. Unlike Rooms 2 and 5, no tested starting pose allowed the robot to complete the full evaluation period in this room.

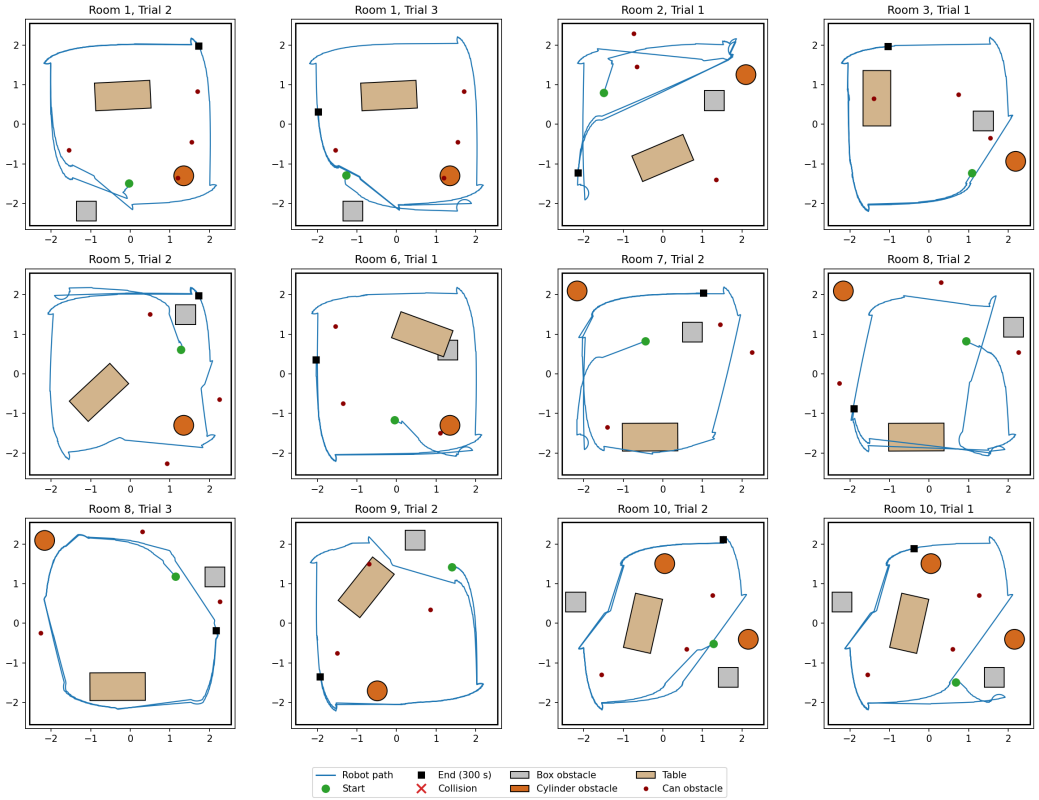


FIGURE 6.1: Successful trajectories from nine test rooms

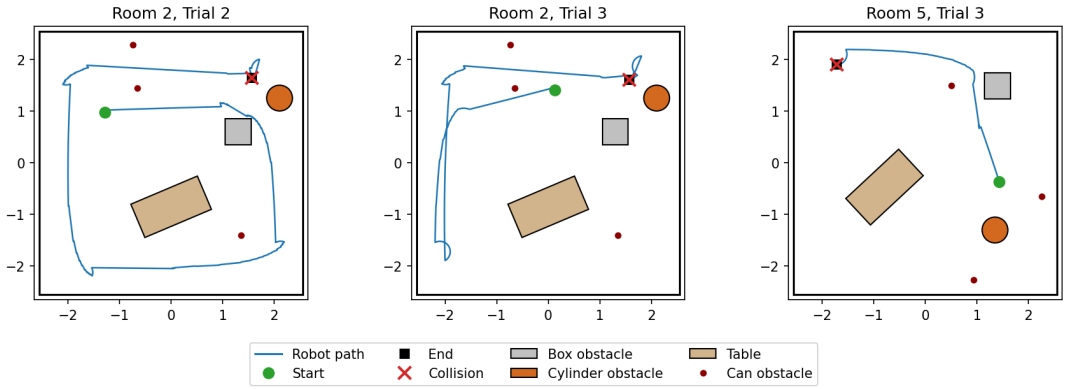


FIGURE 6.2: Failed trajectories in Rooms 2 and 5

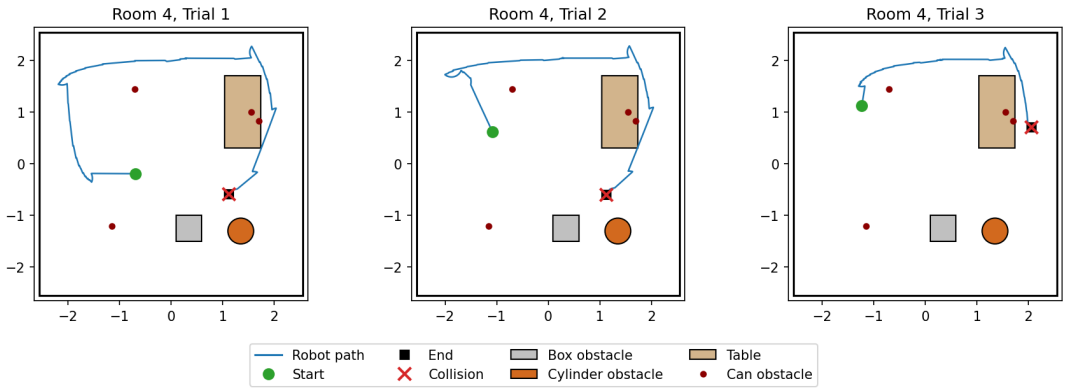


FIGURE 6.3: Failed trajectories in Room 4

6.4 Discussion

The learned policy generalised to unseen environments and completed 24 of the 30 trials successfully. However, performance varied with the room geometry and initial pose. A collision was recorded when the minimum measured depth fell below 0.20 m, indicating entry into the critical range rather than necessarily physical contact.

Both collisions in two unsuccessful trials in Room 2 occurred in the upper-right area of the room, as shown in Figure 6.2. The trajectories suggest that the robot approached the right wall and began turning right. During this movement, the obstacle on its right entered the critical range. At this point, the trial ended before the robot could perform another action. In the successful trial, the robot followed a path closer to the upper wall. This provided enough clearance to complete the right turn without hitting the nearby obstacle.

Room 5 produced one collision near the upper-left wall. The trajectory suggests that the robot turned right and collided with the upper wall, although a left turn would have increased its distance from the wall. The other two trials in this room were successful. Therefore, the failure appears to depend on the approach created by the random starting pose.

Room 4 presents stronger evidence of a systematic difficulty. Trials 1 and 2 ended only about 0.01 m apart near the lower-right cylinder. The robot may have moved too close to the cylinder while turning left. A right turn might have provided more clearance. However, this cannot be confirmed from the trajectory alone. Trial 3 failed at another location in the narrow passage between the vertical table and the right wall. This passage is approximately 0.71 m wide and contains additional nearby objects. A small difference in position or orientation may have caused the obstacle to enter the critical range. The repeated failures show that the policy was not sufficiently robust for this room configuration.

One possible reason is the coarse state representation described in Section 5.1.1. The depth image is reduced to five sectors, and each sector contains only one of three distance levels. This representation does not preserve the exact distance, obstacle shape, or available passage width. Different geometric situations can therefore produce the same state. The state also contains no information about the robot's orientation or previous movement. The policy cannot determine whether the robot is approaching an obstacle or moving away from it.

Right turns accounted for 17.05 % of all evaluation actions, while left turns accounted for only 3.10 %. The Q-table also favours turning right in the symmetric state $(1, 1, 1, 1, 1)$. This imbalance may reduce the ability to respond to obstacle configurations that require a left turn. However, it cannot explain every collision, as demonstrated by the failed trial in Room 5. The model learned 30 of the 32 relevant non-terminal states. This high coverage does not guarantee that every action was explored equally or that every Q-value is reliable.

Overall, the model demonstrates effective obstacle avoidance in most evaluated conditions. Its main limitations appear in narrow passages and in situations that require an early or direction-specific response. The repeated collision locations in Rooms 2 and 4 indicate systematic policy limitations. The mixed results in Room 5 indicate greater sensitivity to the random starting pose.

Chapter 7

Conclusion

7.1 Summary

Before the evaluation, the original URDF model of the Pio robot was corrected and extended. Structural, kinematic, and sensor-related issues were resolved. These changes allowed the robot to operate reliably in Gazebo and provided a stable platform for training and evaluating the Q-learning policy.

The use of five training rooms and random initial poses improved the adaptability of the learned policy. The selected model was evaluated in ten previously unseen room layouts using three starting poses per room. It completed 24 of the 30 trials without a collision, resulting in a collision-free rate of 80 %. The robot successfully navigated most room configurations and completed all three trials in seven rooms. Although the policy selected right turns more frequently than left turns, it was capable of turning in both directions. The trajectories also showed that different starting poses could produce different successful routes through the same room.

7.2 Limitations

One of the main limitations is the uneven generalisation of the learned policy. The robot's performance depends on its starting pose and the obstacle layout. Additional training episodes could expose the robot to more state–action combinations. This could improve states in which turning left is the safer option. However, the training performance did not improve consistently over time. It remained unstable near the end of training. Therefore, increasing the number of episodes would not necessarily produce a better policy.

The evaluated policy works best in simple room layouts with sufficient clearance between obstacles. Narrow passages can cause the robot to enter the critical range while turning. Objects should also not be placed close to walls with a small gap between them. The robot may attempt to enter such a gap even when there is not enough space to pass safely. Low obstacles are another limitation because they may fall outside the camera's field of view.

7.3 Future Work

A next step should be to deploy the current policy on the physical Pio robot and evaluate it in real-world obstacle-avoidance scenarios. The policy completed 24 of 30 trials successfully in previously unseen simulated rooms, providing a basis for testing its performance beyond simulation. Initial experiments should use simple room layouts with sufficient clearance between obstacles. These tests would assess

how sensor noise, control delays, and differences between the simulated and physical robot affect obstacle avoidance.

Further work should focus on improving the state representation and training stability. More distance levels or depth sectors could help the robot recognise narrow passages and select safer turns. The reward function could include a gradual penalty for approaching obstacles. Training should include more varied obstacle layouts and starting poses. The hyperparameters in Table 5.2 can serve as a baseline, but further tuning may be required.

Bibliography

[1] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proceedings of the 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, Sep. 2004, pp. 2149–2154. DOI: 10.1109/IROS.2004.1389727

[2] B. P. Gerkey, R. T. Vaughan, and A. Howard, "The Player/Stage project: Tools for multi-robot and distributed sensor systems," in *Proceedings of the International Conference on Advanced Robotics*, Coimbra, Portugal, Jun. 2003, pp. 317–323.

[3] A. R. Dooraki and D.-J. Lee, "An end-to-end deep reinforcement learning-based intelligent agent capable of autonomous exploration in unknown environments," *Sensors*, vol. 18, no. 10, p. 3575, Oct. 2018. DOI: 10.3390/s18103575

[4] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992. DOI: 10.1007/BF00992698

[5] C.-H. Nguyen, Q.-A. Vu, K.-K. P. Cong, and T.-V. Dang, "Optimal obstacle avoidance strategy using deep reinforcement learning based on stereo camera," *MM Science Journal*, pp. 7556–7561, Oct. 2024. DOI: 10.17973/MMSJ.2024_10_202407 8

[6] L. Kunze, T. Roehm, and M. Beetz, "Towards semantic robot description languages," in *Proceedings of the 2011 IEEE International Conference on Robotics and Automation*, Shanghai, China: IEEE, May 2011, pp. 5589–5595. DOI: 10.1109/ICRA.2011.5980170

[7] D. Tola and P. Corke, "Understanding URDF: A dataset and analysis," *IEEE Robotics and Automation Letters*, vol. 9, no. 5, pp. 4479–4486, May 2024. DOI: 10.1109/LRA.2024.3381482

[8] W. Qian et al., "Manipulation task simulation using ROS and Gazebo," in *Proceedings of the 2014 IEEE International Conference on Robotics and Biomimetics*, Bali, Indonesia: IEEE, Dec. 2014, pp. 2594–2598. DOI: 10.1109/ROBIO.2014.7090732

[9] Open Source Robotics Foundation, *Tutorial: Using a URDF in Gazebo*, https://classic.gazebosim.org/tutorials?tut=ros_urdf, Accessed: July 2026, 2014.

[10] M. Ivanou, S. Mikhel, and S. Savin, "Robot description formats and approaches: Review," in *Proceedings of the 2021 International Conference on Nonlinearity, Information and Robotics*, IEEE, 2021, pp. 1–5. DOI: 10.1109/NIR52917.2021.9666120

[11] A. Afzal, D. S. Katz, C. L. Goues, and C. S. Timperley, "Simulation for robotics test automation: Developer perspectives," in *Proceedings of the 2021 14th IEEE Conference on Software Testing, Verification and Validation*, IEEE, Apr. 2021, pp. 263–274. DOI: 10.1109/ICST49551.2021.00036

[12] M. Quigley et al., "ROS: An open-source Robot Operating System," in *IEEE ICRA Workshop on Open Source Software*, vol. 3, 2009, p. 5.

[13] M. Quigley, B. Gerkey, and W. D. Smart, *Programming Robots with ROS: A Practical Introduction to the Robot Operating System*, 1st ed. Sebastopol, California: O'Reilly Media, Inc., Dec. 2015, ISBN: 978-1-4493-2389-9.

[14] Open Source Robotics Foundation, *Tutorial: Gazebo plugins in ROS*, https://classic.gazebosim.org/tutorials?tut=ros_plugins, Accessed: July 2026, 2014.

[15] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," *Robotics Research: The 14th International Symposium ISRR*, pp. 3–19, 2011. DOI: 10.1007/978-3-642-19457-3_1

[16] G. Khekare and S. Sheikh, "Autonomous navigation using deep reinforcement learning in ROS," *International Journal of Artificial Intelligence and Machine Learning (IJAIML)*, vol. 11, no. 2, pp. 63–70, 2021. DOI: 10.4018/IJAIML.20210701.oa4

[17] Z. Wei, S. Wang, K. Chen, and F. Wang, "ROS-based navigation and obstacle avoidance: A study of architectures, methods, and trends," *Sensors*, vol. 25, no. 14, p. 4306, 2025. DOI: 10.3390/s25144306

[18] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, Second. MIT Press, 2018.

[19] A. B. İnner and M. E. Chachoua, "Depth matters: Geometry-aware RGB-D-based transformer-enabled deep reinforcement learning for mapless navigation," *Applied Sciences*, vol. 16, no. 3, p. 1242, 2026. DOI: 10.3390/app16031242

[20] F. Yang, P. Frivik, D. Hoeller, C. Wang, C. Cadena, and M. Hutter, "Spatially-enhanced recurrent memory for long-range mapless navigation via end-to-end reinforcement learning," *The International Journal of Robotics Research*, vol. 45, no. 11, 2026, First published online December 13, 2025. DOI: 10.1177/02783649251401926

[21] Q. Shen, Z. Wang, Y. Feng, and K. Xu, "ACR-Nav: Localization-free corridor navigation via action-conditioned scalar-range evolution," *Machines*, vol. 14, no. 9, p. 1026, 2026. DOI: 10.3390/machines14091026

[22] J. Xin, T. Xu, H. Zhu, and Y. Liu, "A goal-guided dual-policy deep reinforcement learning framework for multi-robot navigation," *IEEE Transactions on Artificial Intelligence*, 2026, Early access. DOI: 10.1109/TAI.2026.3732286

[23] H. D. S. Himaruwan, S. M. B. P. Samarakoon, M. A. V. J. Muthugala, M. S. Ahamed, and M. R. Elara, "Efficient coverage path planning for a decontamination robot in unknown environments using reinforcement learning," *IEEE Access*, 2026, Early access. DOI: 10.1109/ACCESS.2026.3731209

[24] C.-T. Vu and Y.-C. Liu, "Toward adaptive interaction strategies for human companion robot via deep reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 2026, Early access. DOI: 10.1109/TSMC.2026.3726038

[25] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1–40, 2016. DOI: 10.48550/arXiv.1504.00702

[26] X. Tang, T. Liu, X. Wang, A. Sładkowski, J. Zheng, and Z. Baigunchekov, "Target localization for robotic arm grasping based on deep reinforcement learning," in *2026 8th International Conference on Electronic Engineering and Informatics (EEI)*, 2026, pp. 707–710. DOI: 10.1109/EEI70303.2026.11640700

[27] D. Han, B. Mulyana, V. Stankovic, and S. Cheng, "A survey on deep reinforcement learning algorithms for robotic manipulation," *Sensors*, vol. 23, no. 7, p. 3762, 2023. DOI: 10.3390/s23073762

[28] D. Kalashnikov et al., "Scalable deep reinforcement learning for vision-based robotic manipulation," in *Proceedings of the Conference on Robot Learning (CoRL)*, 2018, pp. 651–673.

[29] T. Johannink et al., "Residual reinforcement learning for robot control," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 6023–6029. DOI: 10.48550/arXiv.1812.03201

[30] Y. Chebotar, K. Hausman, M. Zhang, G. Sukhatme, S. Schaal, and S. Levine, "Combining model-based and model-free updates for trajectory-centric reinforcement learning," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 703–711. DOI: 10.48550/arXiv.1703.03078

[31] J. Luo, C. Xu, J. Wu, and S. Levine, "Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning," *Science Robotics*, vol. 10, no. 105, eads5033, 2025. DOI: 10.1126/scirobotics.ads5033

[32] J. Peters and S. Schaal, "Reinforcement learning of motor skills with policy gradients," *Neural Networks*, vol. 21, no. 4, pp. 682–697, 2008. DOI: 10.1016/j.neunet.2008.02.003

[33] Z. Zhang, J. Hong, A. M. Soufi Enayati, and H. Najjaran, "Using implicit behavior cloning and dynamic movement primitive to facilitate reinforcement learning for robot motion planning," *IEEE Transactions on Robotics*, vol. 40, pp. 4733–4749, 2024. DOI: 10.1109/TRO.2024.3468770

[34] J. B. Alves, N. Lau, and F. Silva, "Curriculum-guided skill learning for long-horizon robot manipulation tasks," *Robotics and Autonomous Systems*, vol. 192, p. 105 032, 2025. DOI: 10.1016/j.robot.2025.105032

[35] J. Tan et al., "Sim-to-real: Learning agile locomotion for quadruped robots," in *Proceedings of Robotics: Science and Systems (RSS)*, 2018. DOI: 10.15607/RSS.2018.XIV.010

[36] D. Hoeller, N. Rudin, D. Sako, and M. Hutter, "ANYmal parkour: Learning agile navigation for quadrupedal robots," *Science Robotics*, vol. 9, no. 88, eadi7566, 2024. DOI: 10.1126/scirobotics.adi7566